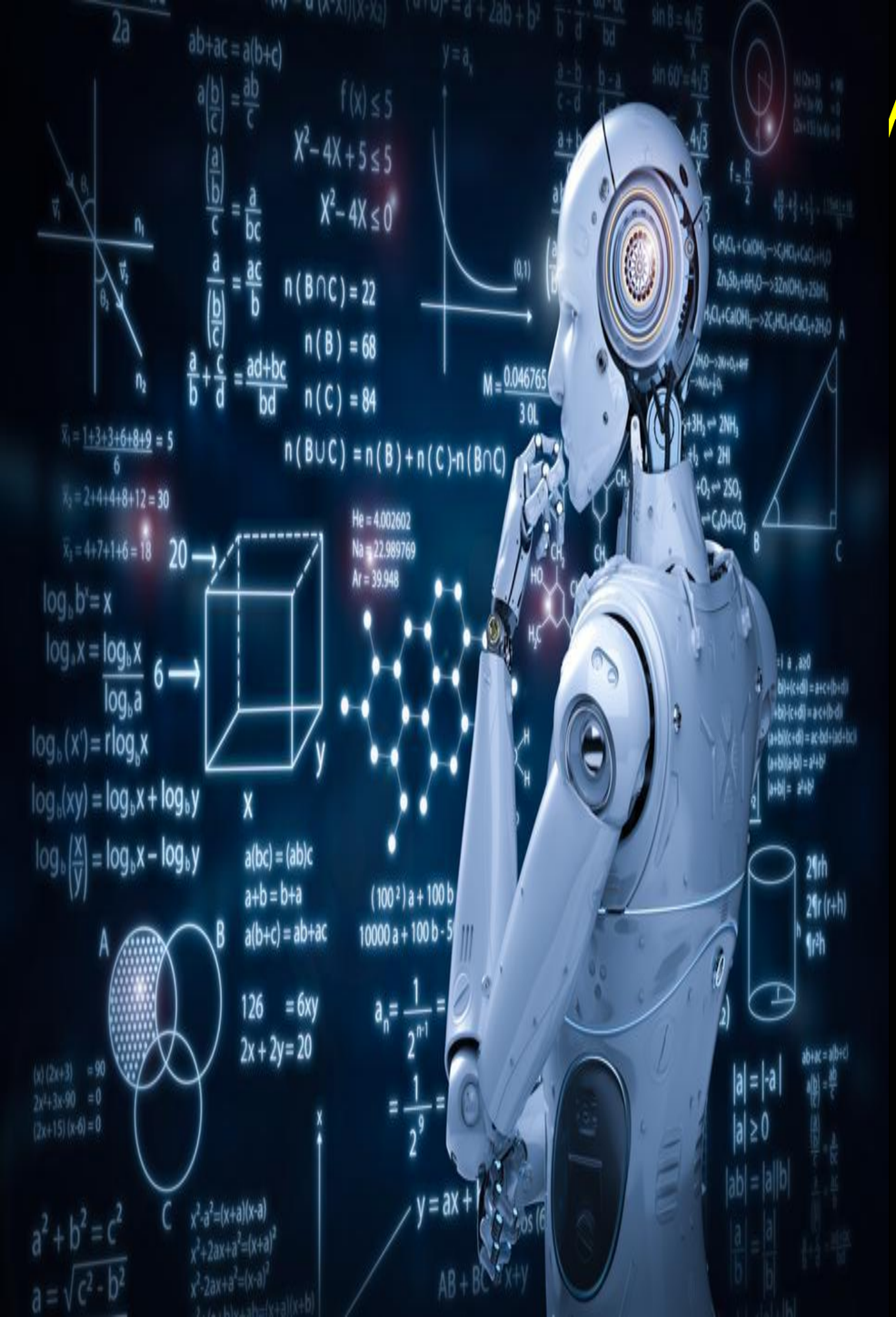
PGCP-AI

PRACTICAL MACHINE LEARNING

Day 11 : SVM

Dr. Kiran Waghmare
CDAC Mumbai



Agenda

- SVM
- SVM – linear
- SVM -- Kernel

SVM : Introduction

A **Support Vector Machine (SVM)** is a supervised machine learning algorithm used for **classification** and **regression**, though it's most commonly applied to classification problems.

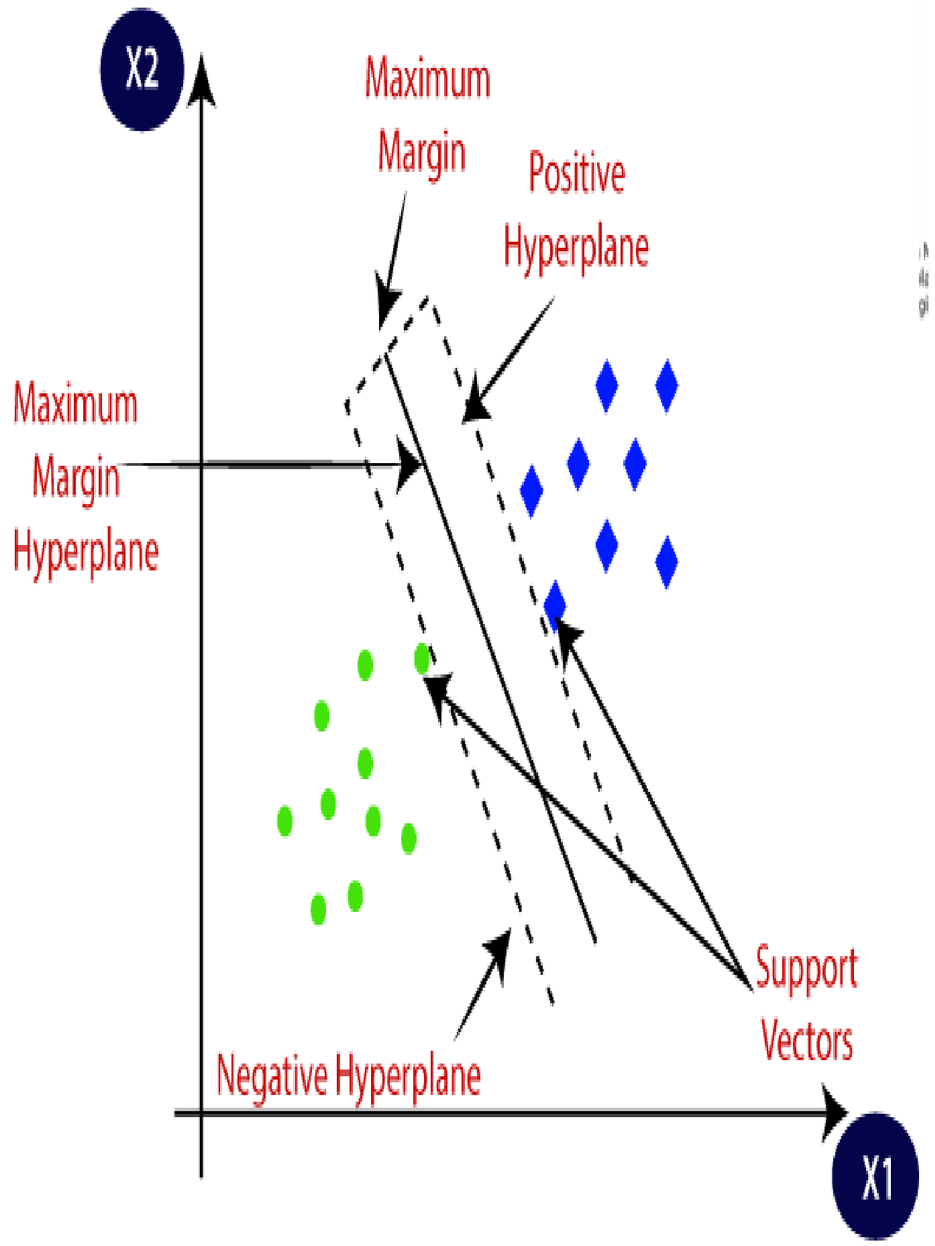
Core idea

SVM tries to find the **best boundary (hyperplane)** that separates data points of different classes.

- In 2D → a line
- In 3D → a plane
- In higher dimensions → a hyperplane

The goal is not just separation, but **maximum separation margin**.

Idea : *Draw the widest possible boundary between classes while touching only the most critical points.*



Key concept: Margin

The **margin** is the distance between the hyperplane and the nearest data points from each class.

- SVM chooses the hyperplane that **maximizes this margin**
- The closest points are called **support vectors**
- These points "support" or define the boundary

Mathematical intuition

For a linear SVM, the decision boundary is:

$$w \cdot x + b = 0$$

- w: weight vector (direction of boundary)
- b: bias (offset)
- x: input features

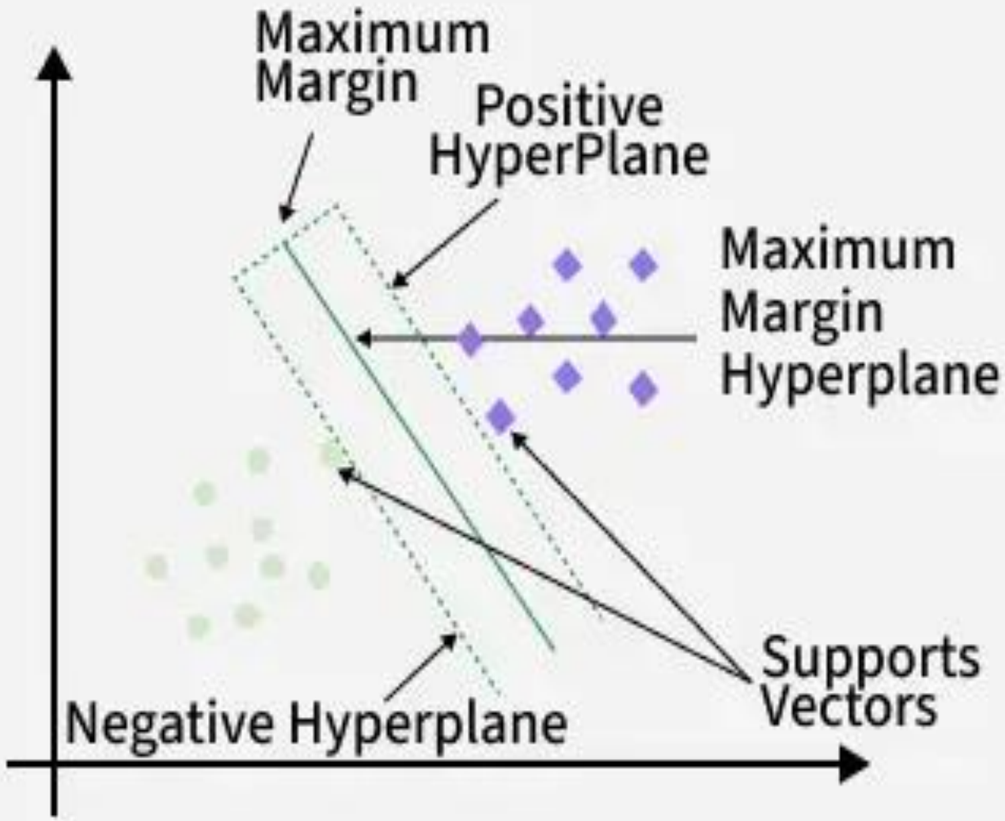
Classification rule:

- If $w \cdot x + b \geq 0 \rightarrow$ Class 1
- Else $\rightarrow$ Class 2

Support Vectors & Hyperplane

- Support Vectors are the closest data points to the hyperplane that define the class boundary.

- A hyperplane is a plane that separates different classes.



Support Vector Machine Algorithm

- **Goal :**
  - The goal of the SVM algorithm is to create the best line or decision boundary that can segregate n-dimensional space into classes so that we can easily put the new data point in the correct category in the future.
  - This best decision boundary is called **a hyperplane**
  - SVM chooses the extreme points/vectors that help in creating the hyperplane
  - These extreme cases are called as **support vectors**

and hence algorithm is termed as Support Vector Machine

Why SVM works well

- Effective in **high-dimensional spaces**
- Works well when **number of features > number of samples**
- Robust to **overfitting** (especially with proper kernel and C)

Limitations

- Not ideal for **very large datasets** (training is slow)
- Choosing the right **kernel and parameters** can be tricky
- Less interpretable than simpler models like logistic regression

Support Vector Machine

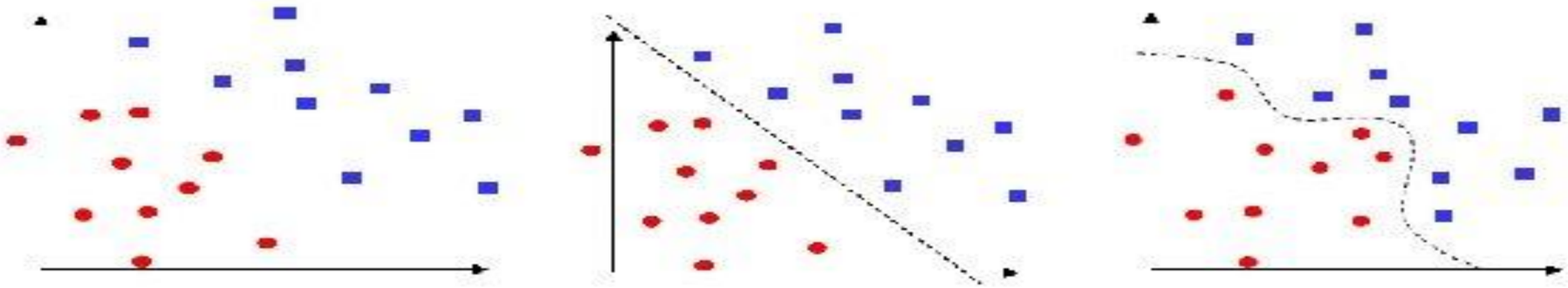
- A new classification method for both linear and nonlinear data
- It uses a nonlinear mapping to transform the original training data into a higher dimension
- With the new dimension, it searches for the linear optimal separating hyperplane (i.e., “decision boundary”)
- With an appropriate nonlinear mapping to a sufficiently high dimension, data from two classes can always be separated by a hyperplane
- SVM finds this hyperplane using support vectors (“essential” training tuples) and margins (defined by the support vectors)

Support Vector Machine (SVM)

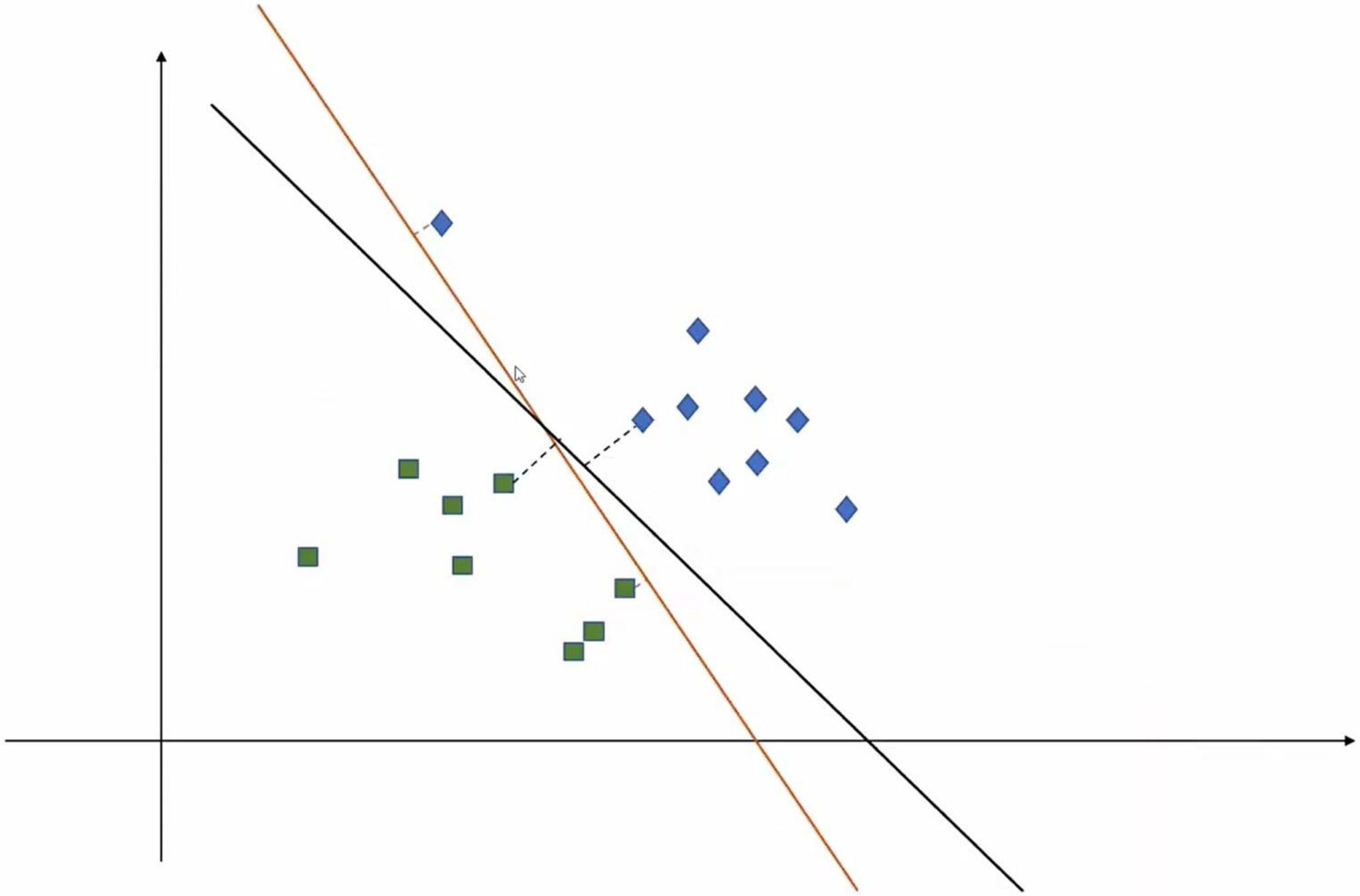
-- classifier, forward neural network, supervised learning

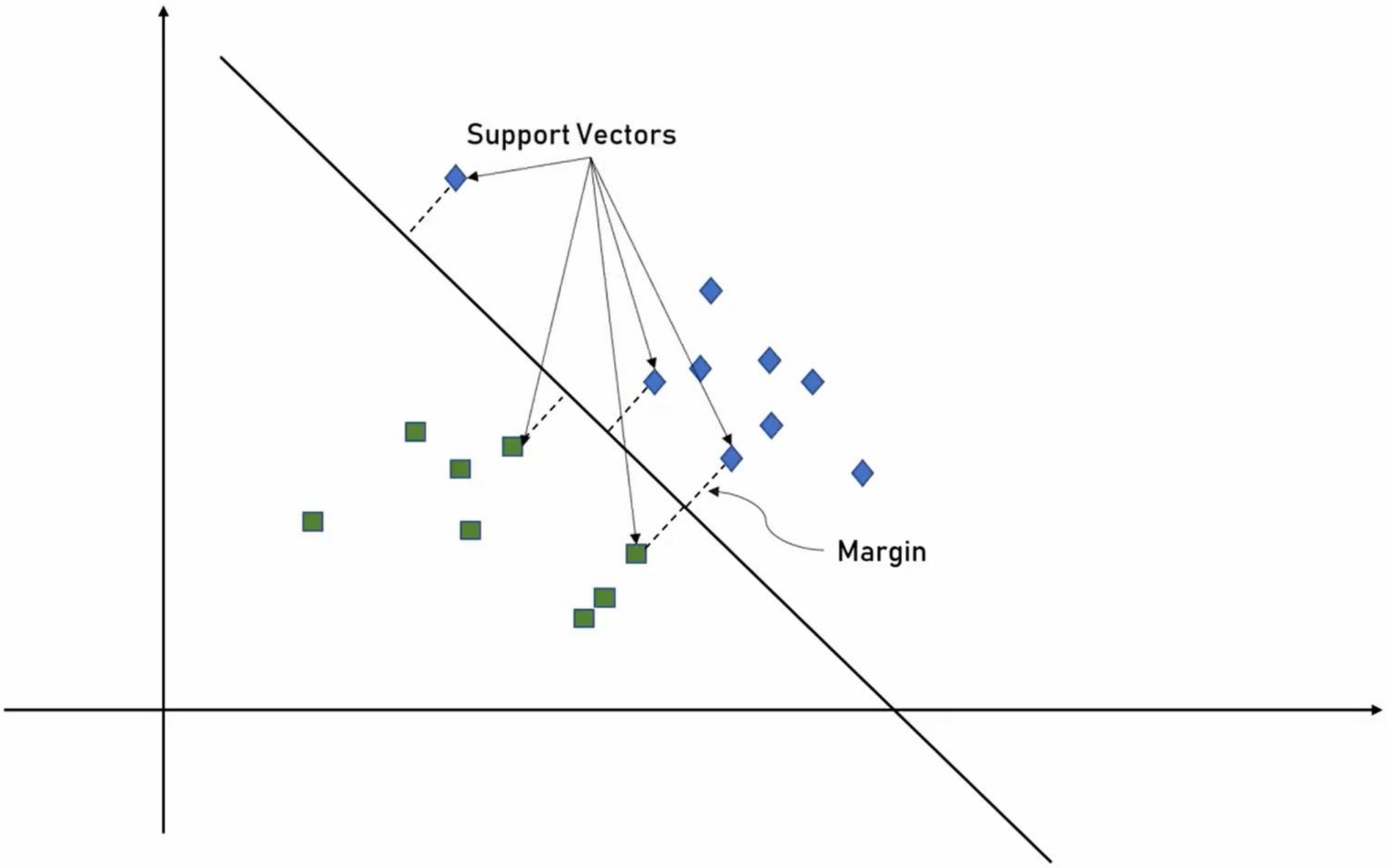
Difficulties with SVM:

i) binary classifier, ii) linearly separable patterns



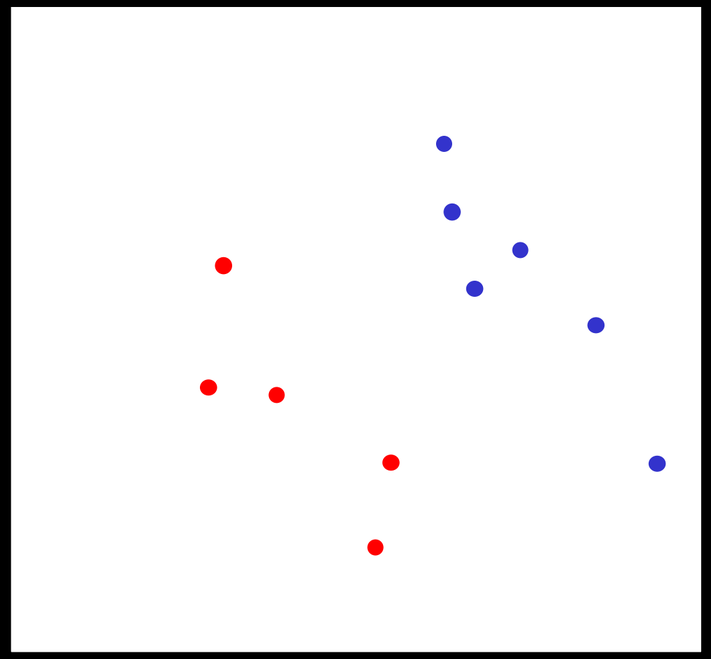
1



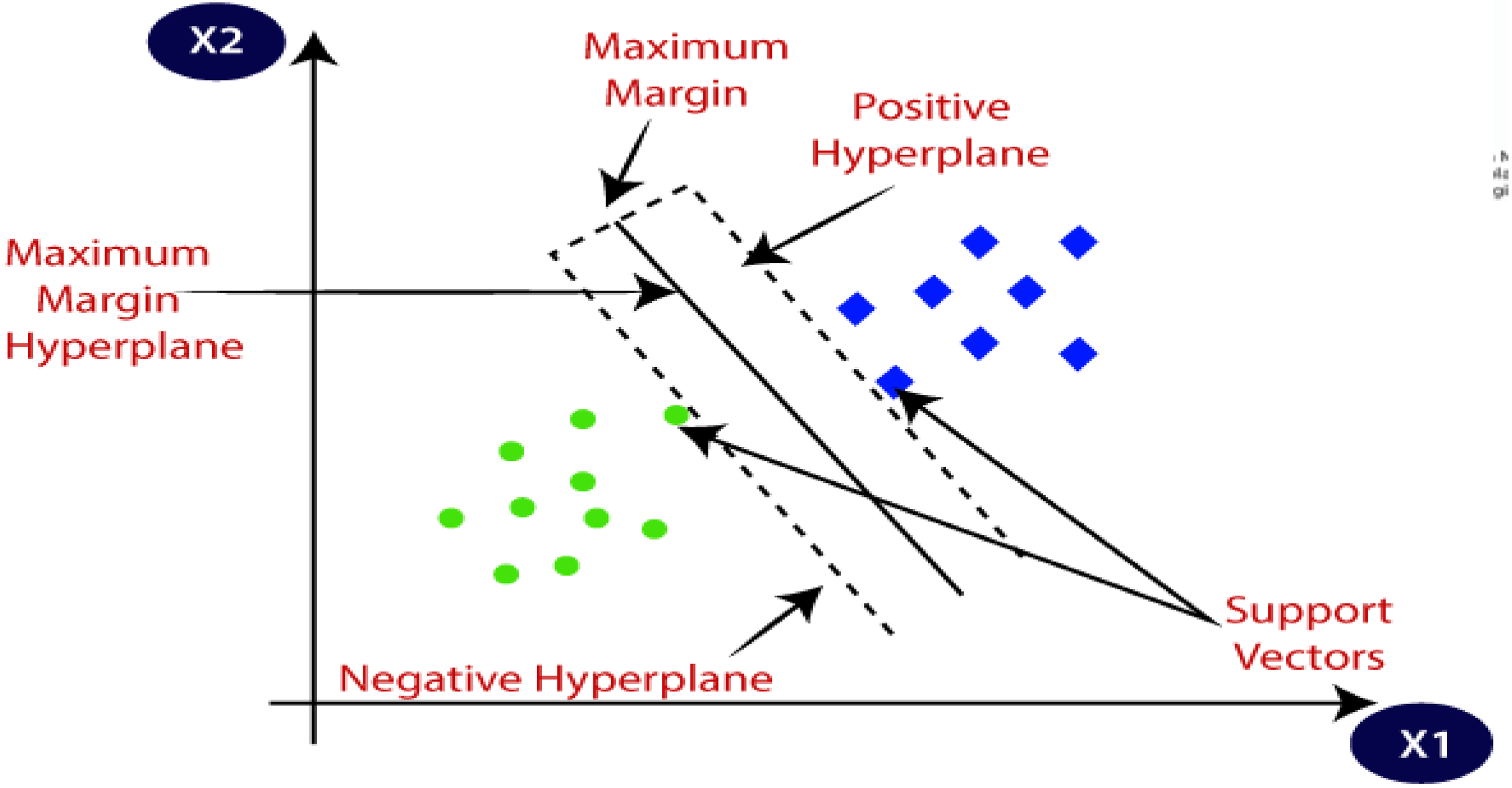

Support Vectors
Margin

Support Vector Machines

- The line that maximizes the minimum margin is a good bet.
  - The model class of "hyper-planes with a margin of m" has a low VC dimension if m is big.
- This maximum-margin separator is determined by a subset of the datapoints.
  - Datapoints in this subset are called "support vectors".
  - It will be useful computationally if only a small fraction of the datapoints are support vectors, because we use the support vectors to decide which side of the separator a test case is on.



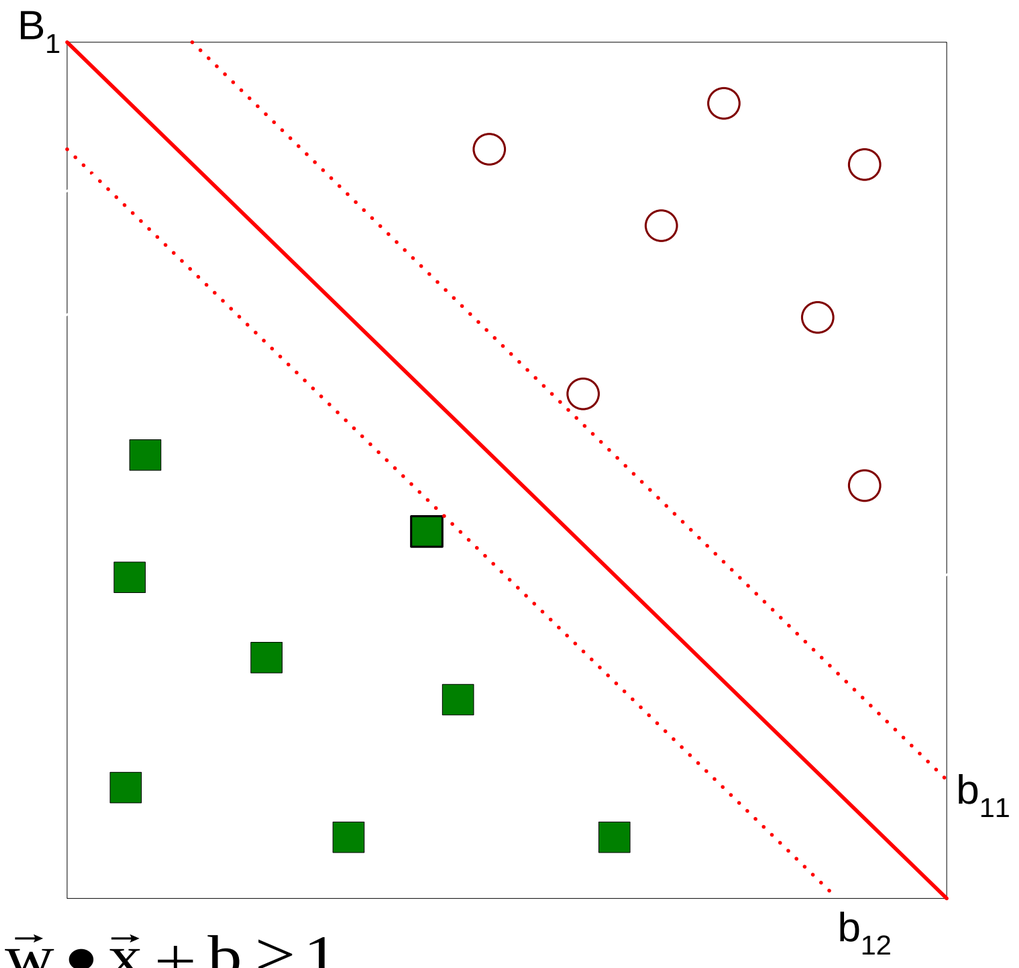
The support vectors are indicated by the circles around them.


X2
Maximum
Margin
Positive
Hyperplane
Maximum
Margin
Hyperplane
Support
Vectors
Negative Hyperplane
X1

Support Vector Machines

$$\vec{w} \bullet \vec{x} + b = 0$$

$$\vec{w} \bullet \vec{x} + b = -1$$



$$\vec{w} \bullet \vec{x} + b = +1$$

$$f(\vec{x}) = \begin{cases} 1 & \text{if } \vec{w} \bullet \vec{x} + b \geq 1 \\ -1 & \text{if } \vec{w} \bullet \vec{x} + b \leq -1 \end{cases}$$

$$\text{Margin} = \frac{2}{\|\vec{w}\|}$$

Training a linear SVM

- To find the maximum margin separator, we have to solve the following optimization problem:

$$\mathbf{w}.\mathbf{x}^c + b > +1 \quad \textit{for positive cases}$$

$$\mathbf{w}.\mathbf{x}^c + b < -1 \quad \textit{for negative cases}$$

$$\textit{and} \quad \|\mathbf{w}\|^2 \ \textit{is as small as possible}$$

- This is tricky but it's a convex problem. There is only one optimum and we can find it without fiddling with learning rates or weight decay or early stopping.
  - Don't worry about the optimization problem. It has been solved. Its called quadratic programming.
  - It takes time proportional to N^2 which is really bad for very big datasets
    - so for big datasets we end up doing approximate optimization!

Testing a linear SVM

- The separator is defined as the set of points for which:

$$\mathbf{w}.\mathbf{x} + b = 0$$

$$so\ if\ \ \mathbf{w}.\mathbf{x}^c + b > 0\ \ say\ its\ a\ \ positive\ \ case$$

$$and\ if\ \ \mathbf{w}.\mathbf{x}^c + b < 0\ \ say\ its\ a\ negative\ \ case$$

Linear SVM

- Linear model:

$$f(\vec{x}) = \begin{cases} 1 & \text{if } \vec{\mathrm{w}} \bullet \vec{\mathrm{x}} + \mathrm{b} \geq 1 \\ -1 & \text{if } \vec{\mathrm{w}} \bullet \vec{\mathrm{x}} + \mathrm{b} \leq -1 \end{cases}$$

- Learning the model is equivalent to determining the values of

  - How to find from training data?

$$\vec{w} \text{ and } b$$

$$\vec{w} \text{ and } b$$

Learning Linear SVM

- Objective is to maximize:

$$\text{Margin} = \frac{2}{\|\vec{w}\|}$$

  - Which is equivalent to minimizing:

$$L(\vec{w}) = \frac{\|\vec{w}\|^2}{2}$$

  - Subject to the following constraints:

$$y_i = \begin{cases} 1 & \text{if } \vec{\mathrm{w}} \bullet \vec{\mathrm{x}}_\mathrm{i} + \mathrm{b} \geq 1 \\ -1 & \text{if } \vec{\mathrm{w}} \bullet \vec{\mathrm{x}}_\mathrm{i} + \mathrm{b} \leq -1 \end{cases}$$

or

$$y_i(\mathrm{w} \bullet \mathrm{x}_i + b) \geq 1, \qquad i = 1,2,\ldots,N$$

  - This is a constrained optimization problem
    - Solve it using Lagrange multiplier method

1) Decision Boundary (Hyperplane)

The separating hyperplane is:

$$w \cdot x + b = 0$$

- w: weight vector (orientation of the boundary)
- b: bias (offset from origin)
- x: input feature vector

This equation defines the boundary between two classes.

---

2) Classification Rule

Prediction is based on the sign:

$$\text{class} = \begin{cases} +1 & \text{if } w \cdot x + b \geq 0 \\ -1 & \text{if } w \cdot x + b < 0 \end{cases}$$

3) Margin and Supporting Hyperplanes

SVM defines two parallel boundaries:

$$w \cdot x + b = 1 \quad \text{and} \quad w \cdot x + b = -1$$

- Points on these lines are **support vectors**
- Distance between them is the **margin**

---

4) Margin Formula

The margin width is:

$$\text{Margin} = \frac{2}{||w||}$$

- Smaller $||w||$ → larger margin
- SVM tries to maximize this

5) Hard Margin Constraints (Perfect Separation)

For perfectly separable data:

$$y_i(w \cdot x_i + b) \geq 1$$

- Ensures all points are correctly classified
- $y_i \in \{+1, -1\}$

---

6) Optimization Objective (Hard Margin)

SVM solves:

$$\min_{w,b} \frac{1}{2}||w||^2$$

subject to the constraint above.

- Minimizing $||w||^2$ → maximizing margin

7) Soft Margin (Non-Separable Case)

When data is not perfectly separable:

$$y_i(w \cdot x_i + b) \geq 1 - \xi_i$$

- ξ_i: slack variable (error allowance)

Updated objective:

$$\min \frac{1}{2}||w||^2 + C \sum_i \xi_i$$

- C: penalty for misclassification

---

8) Key Insight

- Only points satisfying $w \cdot x + b = \pm 1$ matter → **support vectors**
- Other points do not influence the boundary

What if data is not linearly separable?

Real-world data is often not separable by a straight line.

SVM handles this in two ways:

1. Soft margin

Allows some misclassification using a parameter **C**

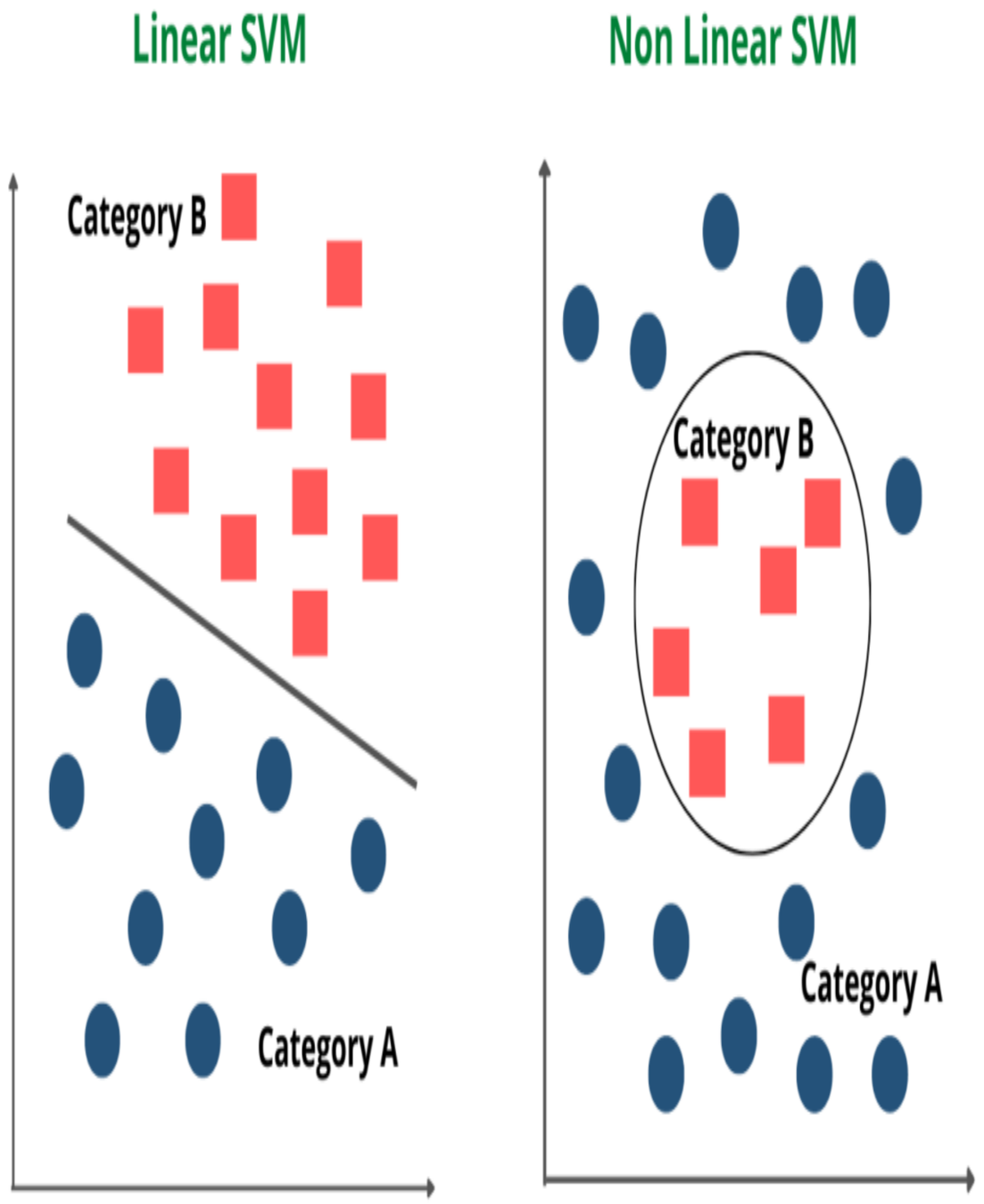
- Large C → fewer errors, but tighter margin
- Small C → more tolerance, wider margin

2. Kernel trick

Transforms data into a higher dimension where separation is possible.

Common kernels:

- Linear kernel
- Polynomial kernel
- RBF (Gaussian) kernel ← most popular
- Sigmoid kernel



- **Kernel Function** is **a method used to take data as input and transform it into the required form of processing data**. “Kernel” is used due to a **set of mathematical functions** used in Support Vector Machine providing the window to manipulate the data.

- **What is kernel in machine learning?**
- In machine learning, a kernel refers to a method that **allows us to apply linear classifiers to non-linear problems** by mapping non-linear data into a higher-dimensional space without the need to visit or understand that higher-dimensional space.

- **What does kernel function do?**
- The kernel function is what is applied on each data instance to **map the original non-linear observations into a higher-dimensional space in which they become separable**.

Types of Kernel Functions

Some commonly used kernels

Polynomial: $K(\mathbf{x},\mathbf{y}) = (\mathbf{x}.\mathbf{y}+1)^p$

Gaussian radial basis function $K(\mathbf{x},\mathbf{y}) = e^{-\|\mathbf{x}-\mathbf{y}\|^2/2\sigma^2}$

Parameters that the user must choose

Neural net: $K(\mathbf{x},\mathbf{y}) = \tanh(k\,\mathbf{x}.\mathbf{y} - \delta)$

For the neural network kernel, there is one "hidden unit" per support vector, so the process of fitting the maximum margin hyperplane decides how many hidden units to use.

What to do if there is no separating plane ?

- **Use a much bigger set of features.**
  - This looks as if it would make the computation slow, but we can use the **"kernel"** trick to make the computation fast even with huge numbers of features.
- **Extend the definition of maximum margin to allow non-separating planes.**
  - This can be done by using "slack" variables

Introducing slack variables

- Slack variables are constrained to be non-negative. When they are greater than zero they allow us to cheat by putting the plane closer to the datapoint than the margin. So we need to minimize the amount of cheating. This means we have to pick a value for lamba (this sounds familiar!)

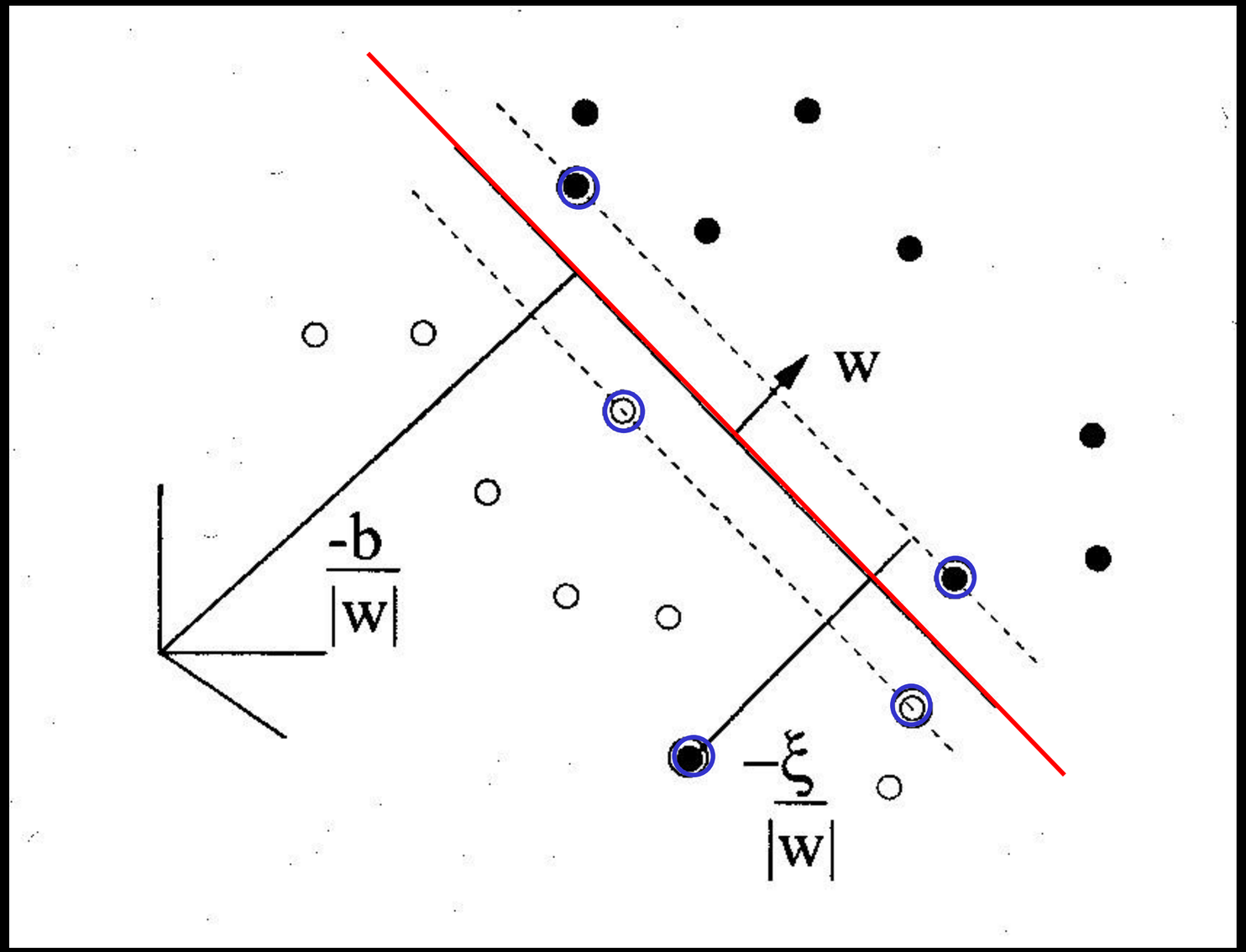
$$\mathbf{w}.\mathbf{x}^c + b \geq +1 - \xi^c \quad \textit{for positive cases}$$

$$\mathbf{w}.\mathbf{x}^c + b \leq -1 + \xi^c \quad \textit{for negative cases}$$

$$\textit{with} \quad \xi^c \geq 0 \quad \textit{for all } c$$

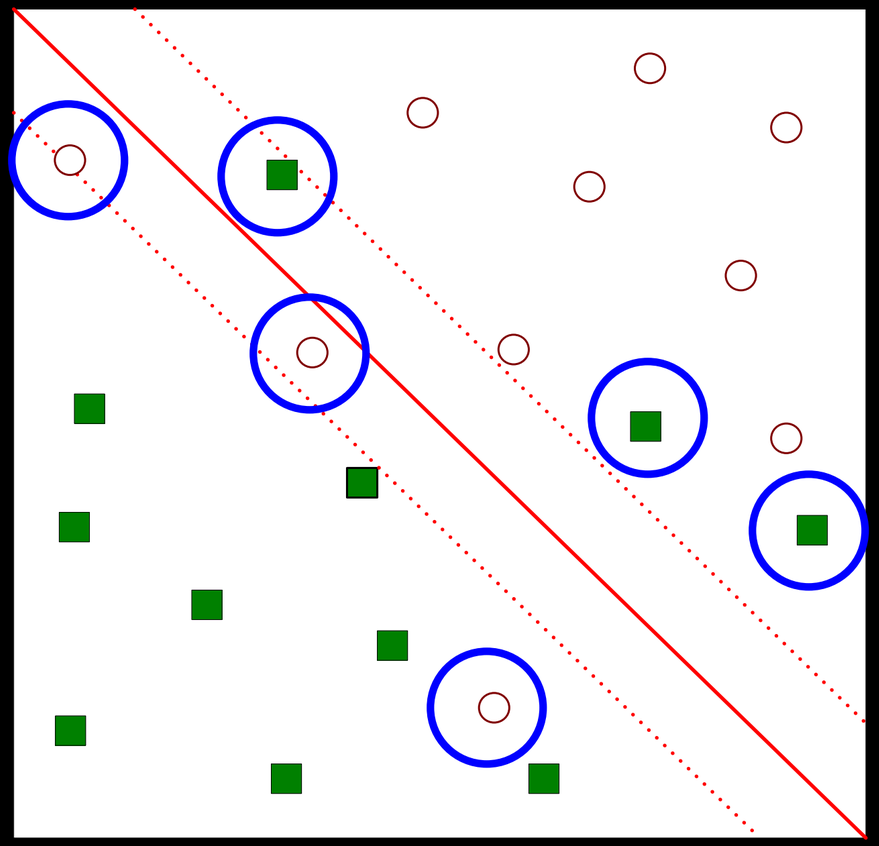
$$\textit{and} \quad \frac{\|\mathbf{w}\|^2}{2} + \lambda \sum_c \xi^c \quad \textit{as small as possible}$$

A picture of the best plane with a slack variable



Support Vector Machines

- What if the problem is not linearly separable?



Support Vector Machines

- What if the problem is not linearly separable?
  - Introduce slack variables
    - Need to minimize:

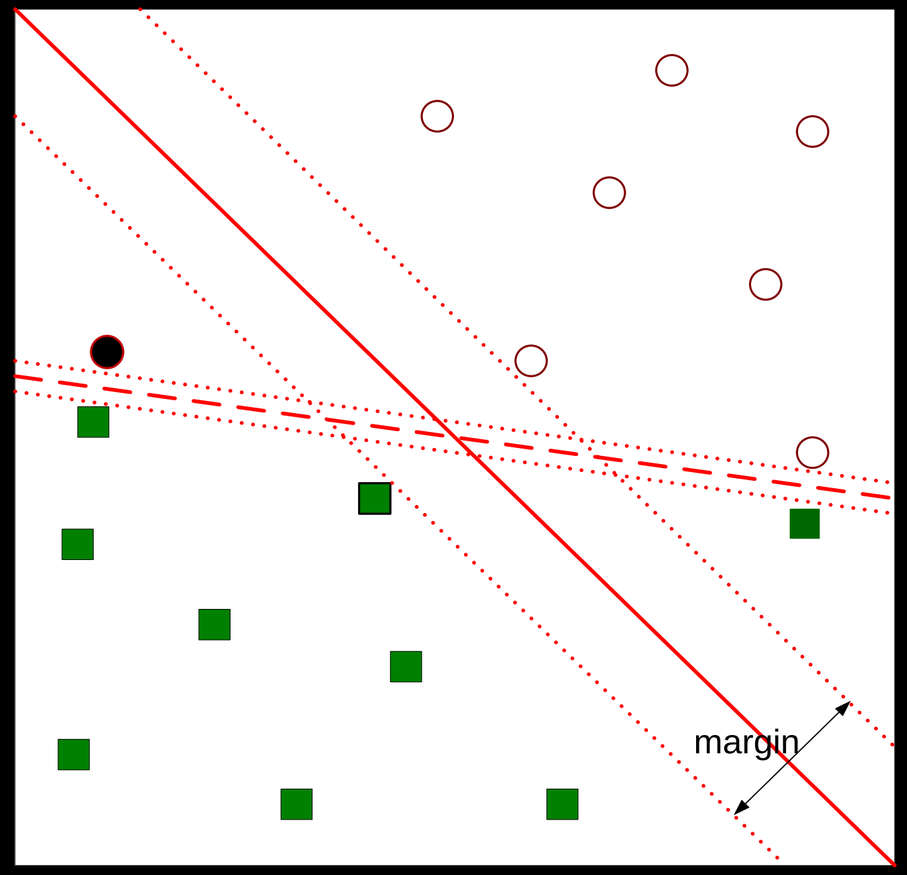
$$L(w) = \frac{\|\vec{w}\|^2}{2} + C\left(\sum_{i=1}^{N} \xi_i^k\right)$$

    - Subject to:

$$y_i = \begin{cases} 1 & \text{if } \vec{w} \bullet \vec{x}_i + b \geq 1 - \xi_i \\ -1 & \text{if } \vec{w} \bullet \vec{x}_i + b \leq -1 + \xi_i \end{cases}$$

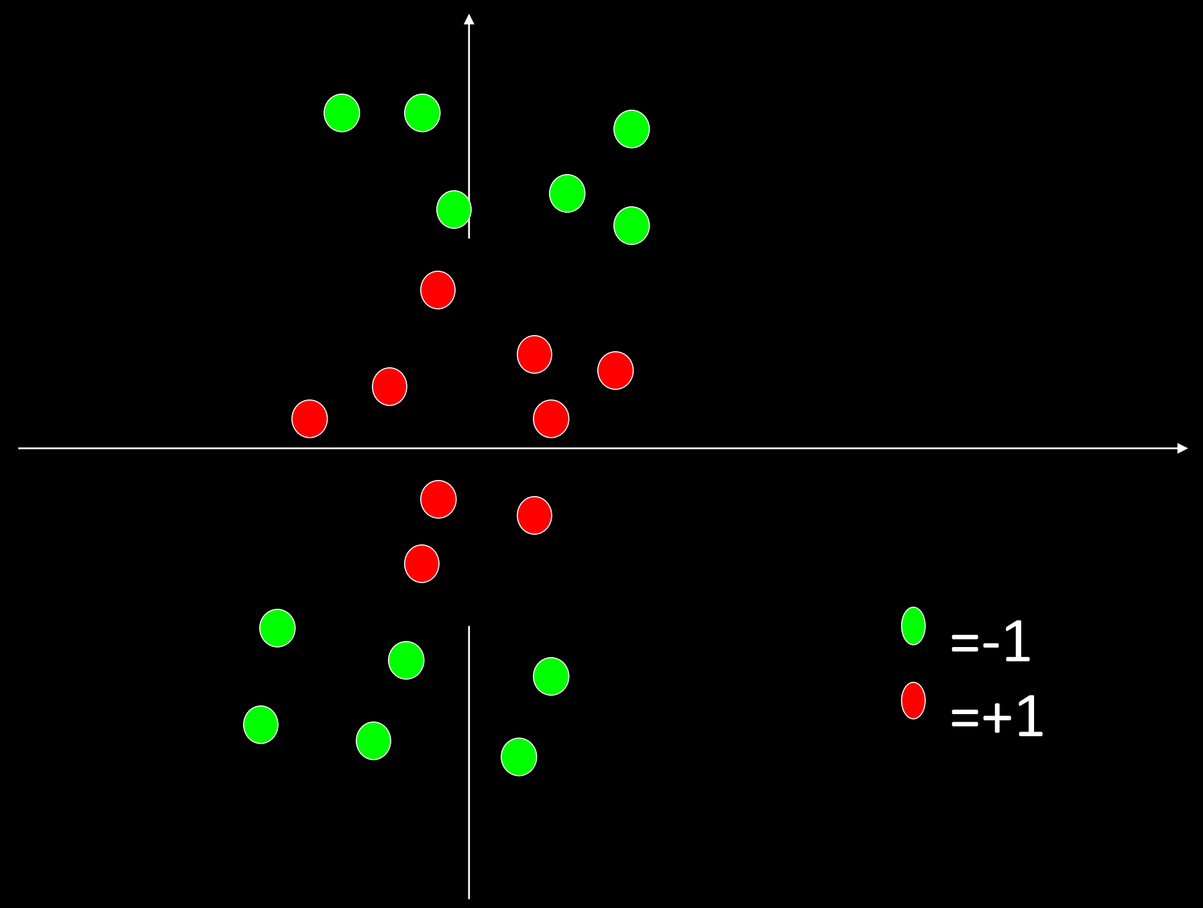
    - If k is 1 or 2, this leads to similar objective function as linear SVM but with different constraints (see textbook)

Support Vector Machines



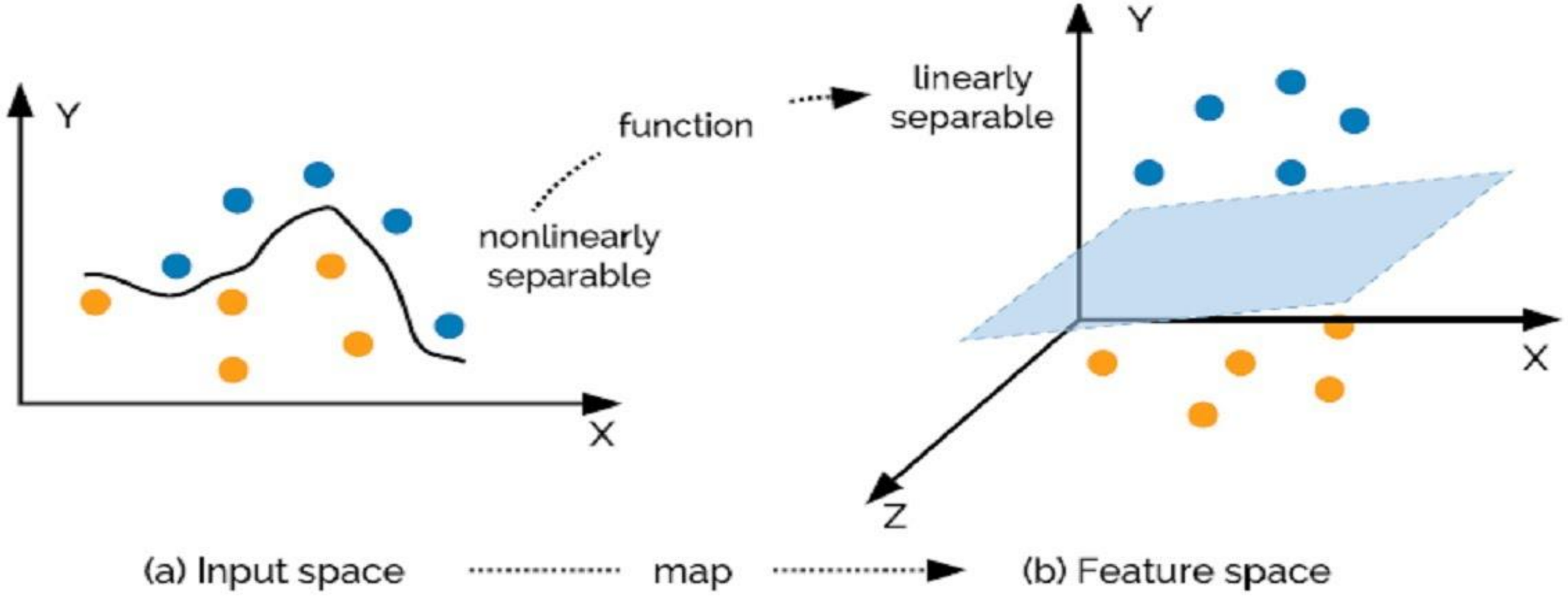
- Find the hyperplane that optimizes both factors

Problems with linear SVM



What if the decision function is not a linear?

Kernal Trick (SVM)…



1) Primal Optimization Problem (Soft Margin)

When data is not linearly separable, SVM solves:

$$\min_{w,b,\xi} \ \frac{1}{2}||w||^2 + C\sum_{i=1}^{n} \xi_i$$

Subject to constraints:

$$y_i\big(w \cdot \phi(x_i) + b\big) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

Terms explained:

- w: weight vector
- b: bias
- ξ_i: slack variables (allow misclassification)
- C: penalty parameter
- $\phi(x)$: mapping to higher-dimensional space

2) Feature Mapping (Non-linearity)

Instead of working in original space:

$$x \to \phi(x)$$

- $\phi(x)$ maps data into a higher dimension
- In that space, data becomes linearly separable

3) Dual Formulation (Key for Kernels)

The problem is converted into its dual form:

$$\max_{\alpha} \ \sum_{i=1}^{n} \alpha_i - \frac{1}{2}\sum_{i,j} \alpha_i \alpha_j y_i y_j K(x_i, x_j)$$

Subject to:

$$0 \leq \alpha_i \leq C, \quad \sum_{i=1}^{n} \alpha_i y_i = 0$$

Terms:

- α_i: Lagrange multipliers
- $K(x_i, x_j)$: kernel function

4) Kernel Function (Core Idea)

Instead of computing:

$$\phi(x_i) \cdot \phi(x_j)$$

We use:

$$K(x_i, x_j)$$

Common kernels:

- Linear: $K(x_i, x_j) = x_i \cdot x_j$
- Polynomial: $K = (x_i \cdot x_j + 1)^d$
- RBF: $K = e^{-\gamma||x_i - x_j||^2}$

This avoids explicit high-dimensional computation.

5) Decision Function

After training, prediction is:

$$f(x) = \sum_{i=1}^{n} \alpha_i y_i K(x_i, x) + b$$

- Only support vectors have $\alpha_i > 0$
- Others do not contribute

6) Classification Rule

$$\text{Class} = \begin{cases} +1 & \text{if } f(x) \geq 0 \\ -1 & \text{otherwise} \end{cases}$$

7) Key Intuition

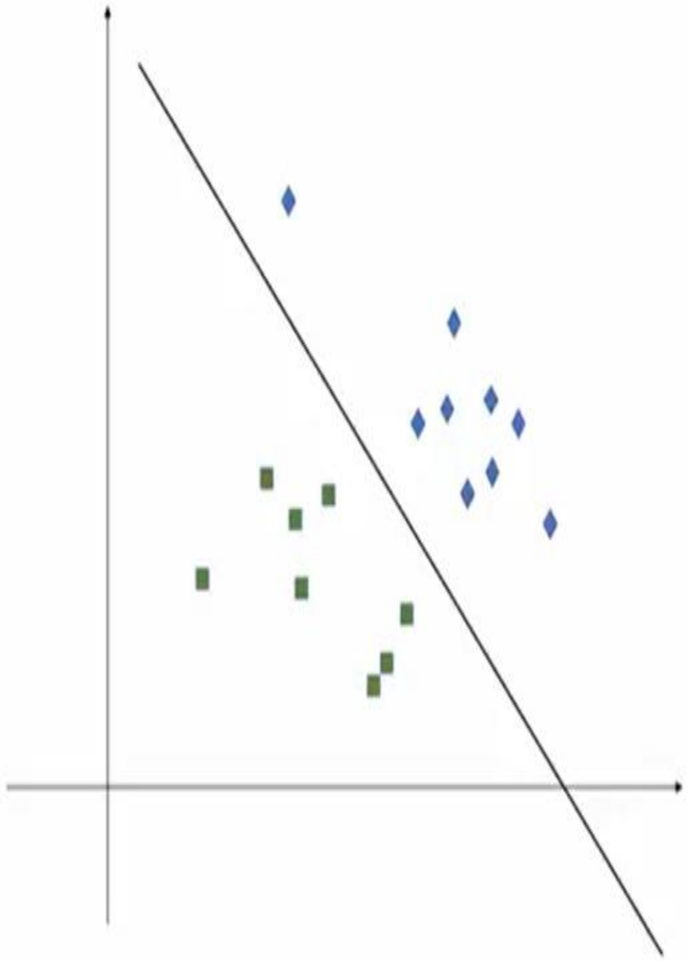
- Data is mapped to a higher-dimensional space
- A linear boundary is found there
- That boundary corresponds to a **non-linear boundary** in original space

Key Pointer

Non-linear SVM equations involve:

1. **Primal problem** → margin + slack variables
2. **Dual problem** → expressed using kernels
3. **Kernel trick** → enables non-linear separation
4. **Decision function** → based only on support vectors

2D



3D

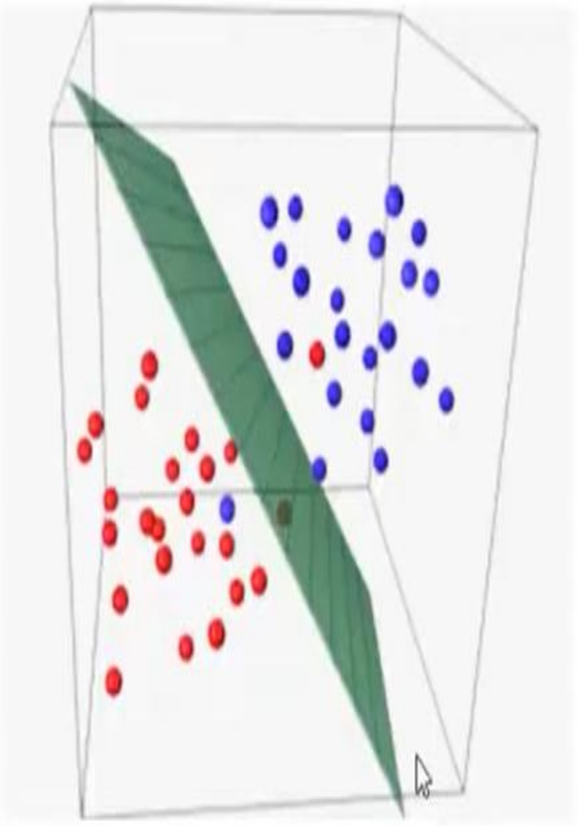
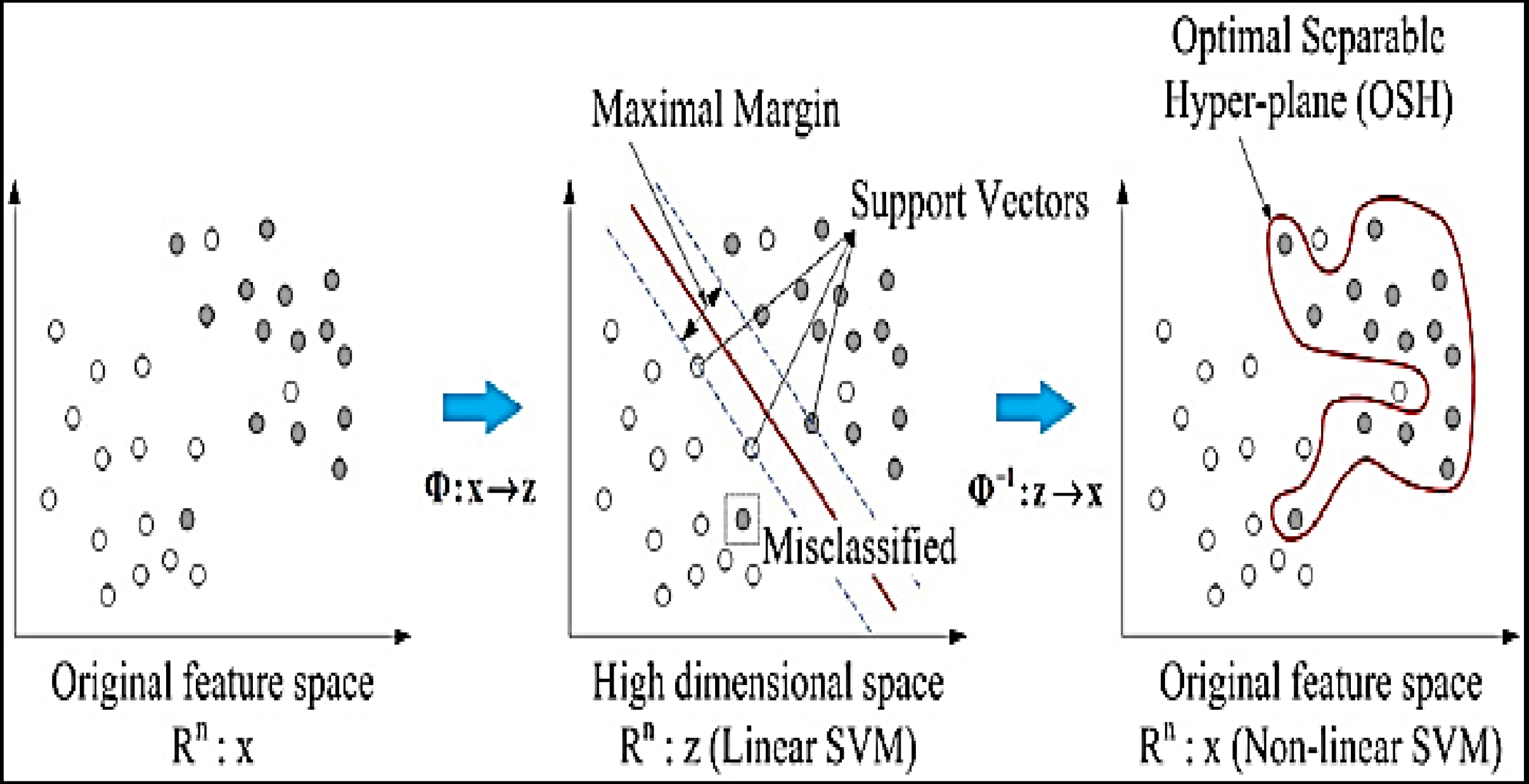
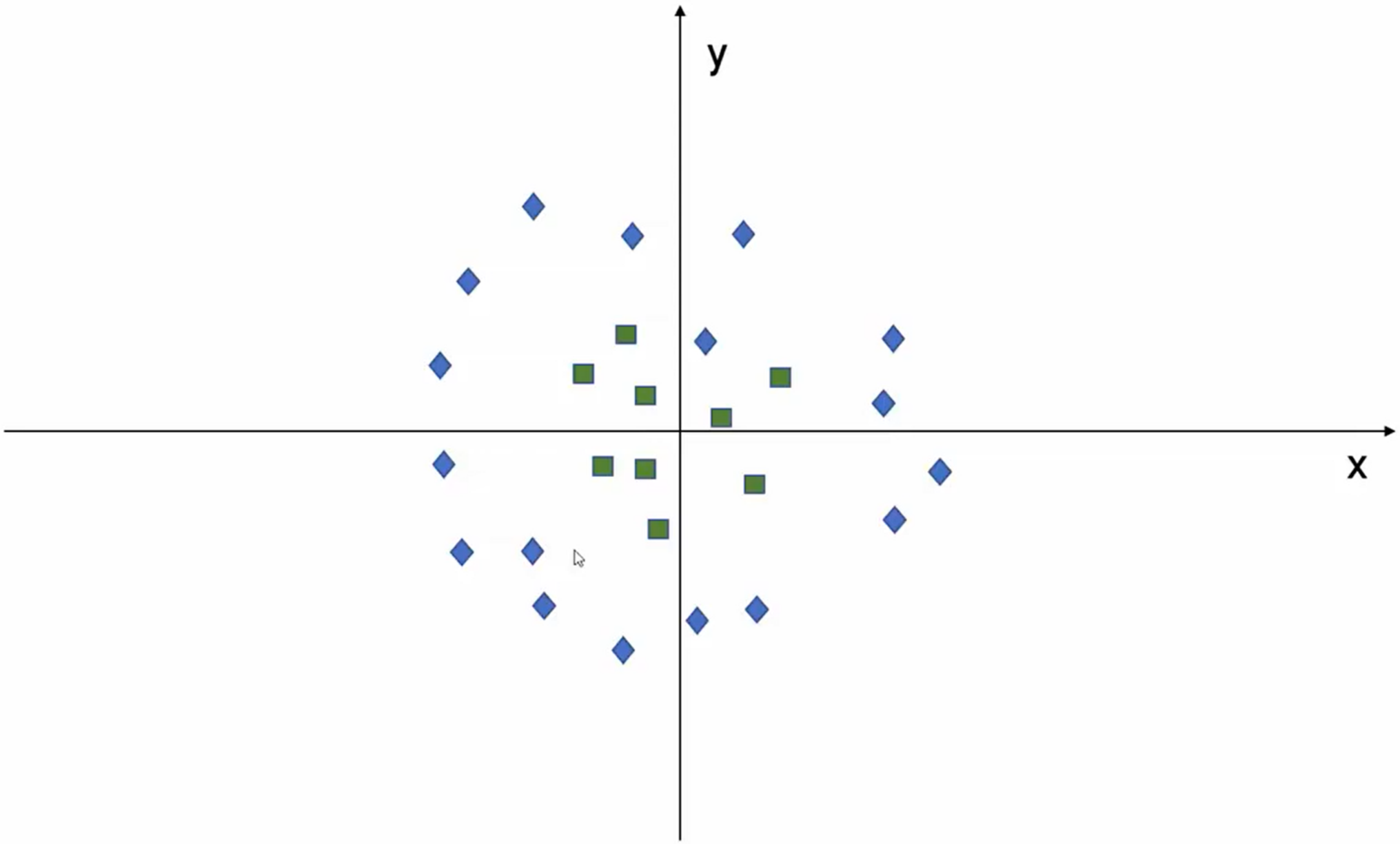


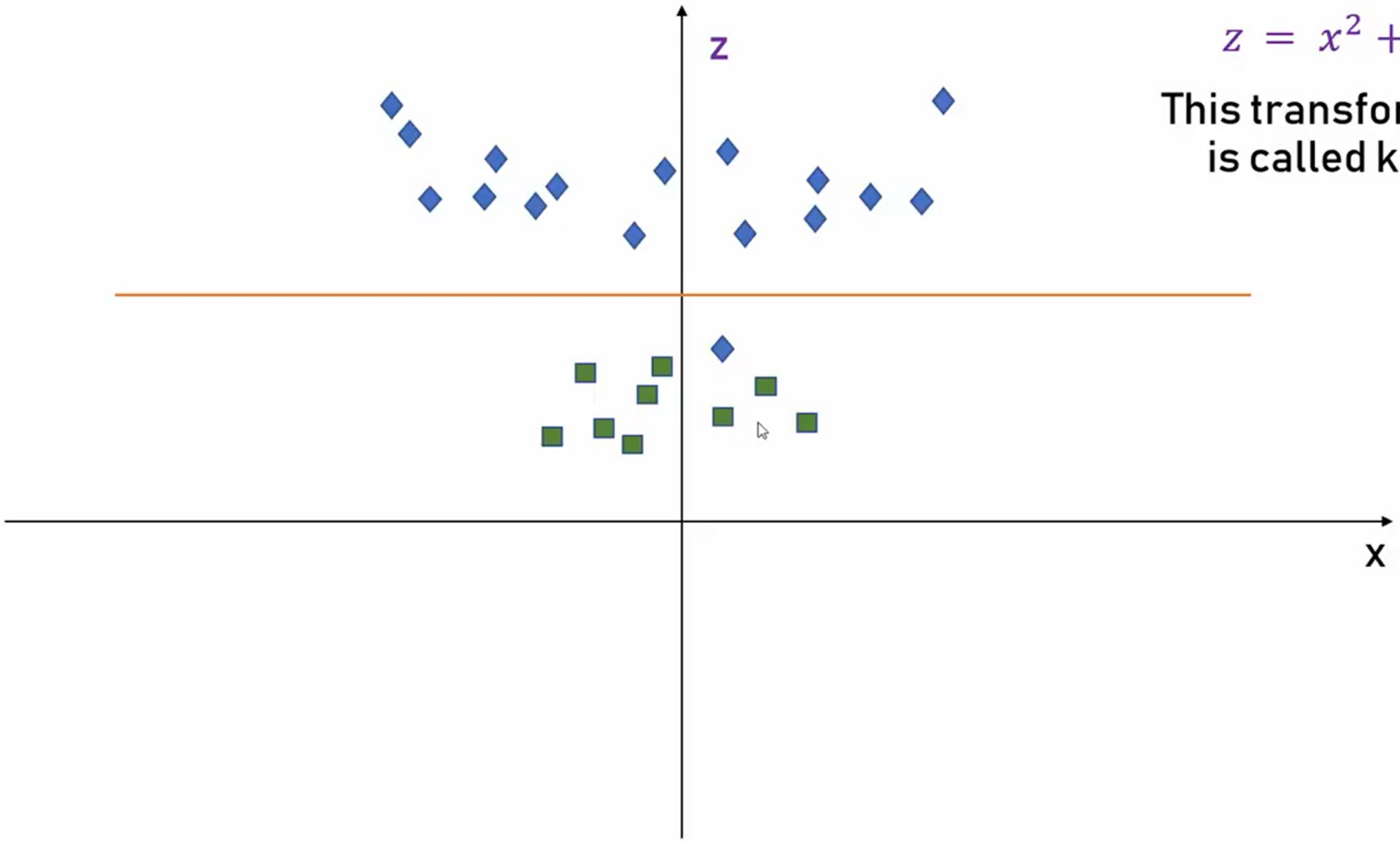
Image Credit: https://appliedmachinelearning.blog

nD


Optimal Separable
Hyper-plane (OSH)
Maximal Margin
Support Vectors
Misclassified
Φ:x→z
Φ⁻¹:z→x
Original feature space
Rⁿ : x
High dimensional space
Rⁿ : z (Linear SVM)
Original feature space
Rⁿ : x (Non-linear SVM)

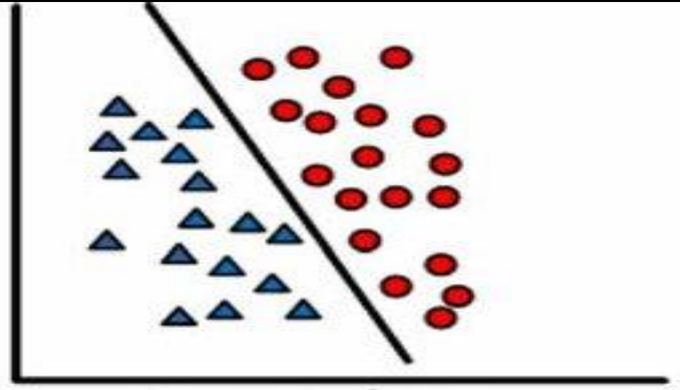
Support vector machine draws a hyper plane in n dimensional space such that it maximizes margin between classification groups


y
x



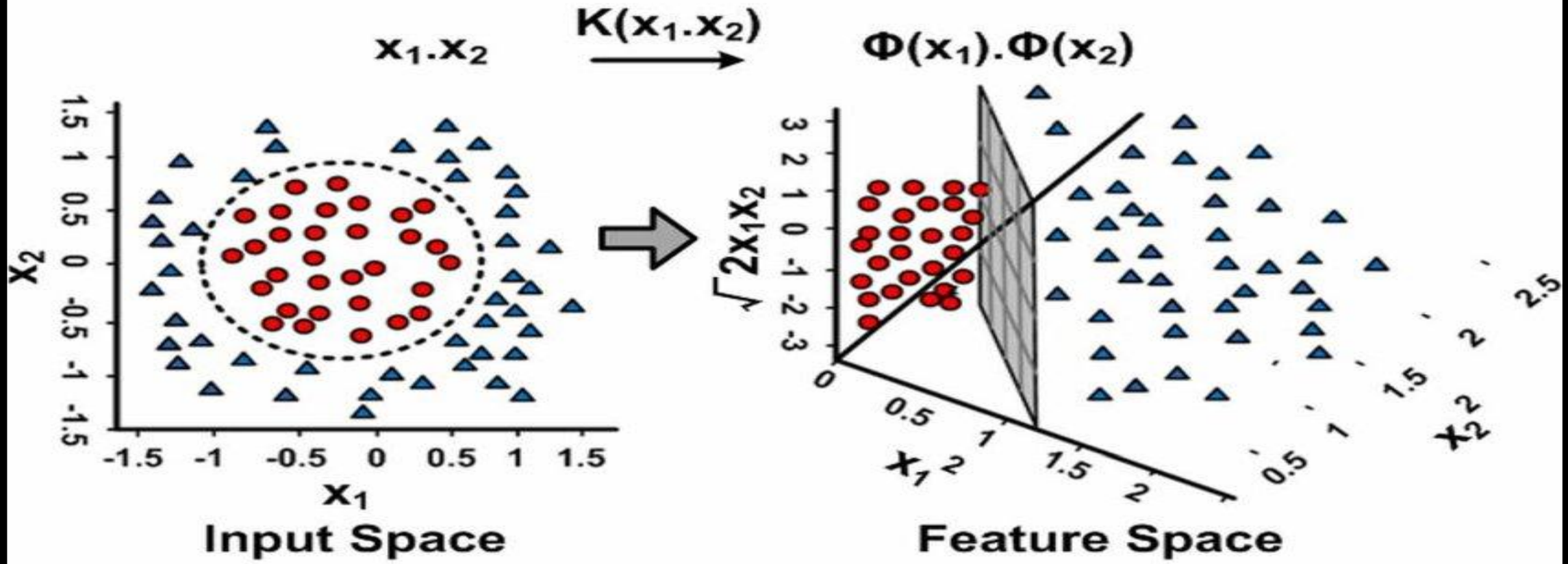
$$z = x^2 + y^2$$

This transformation is called kernel

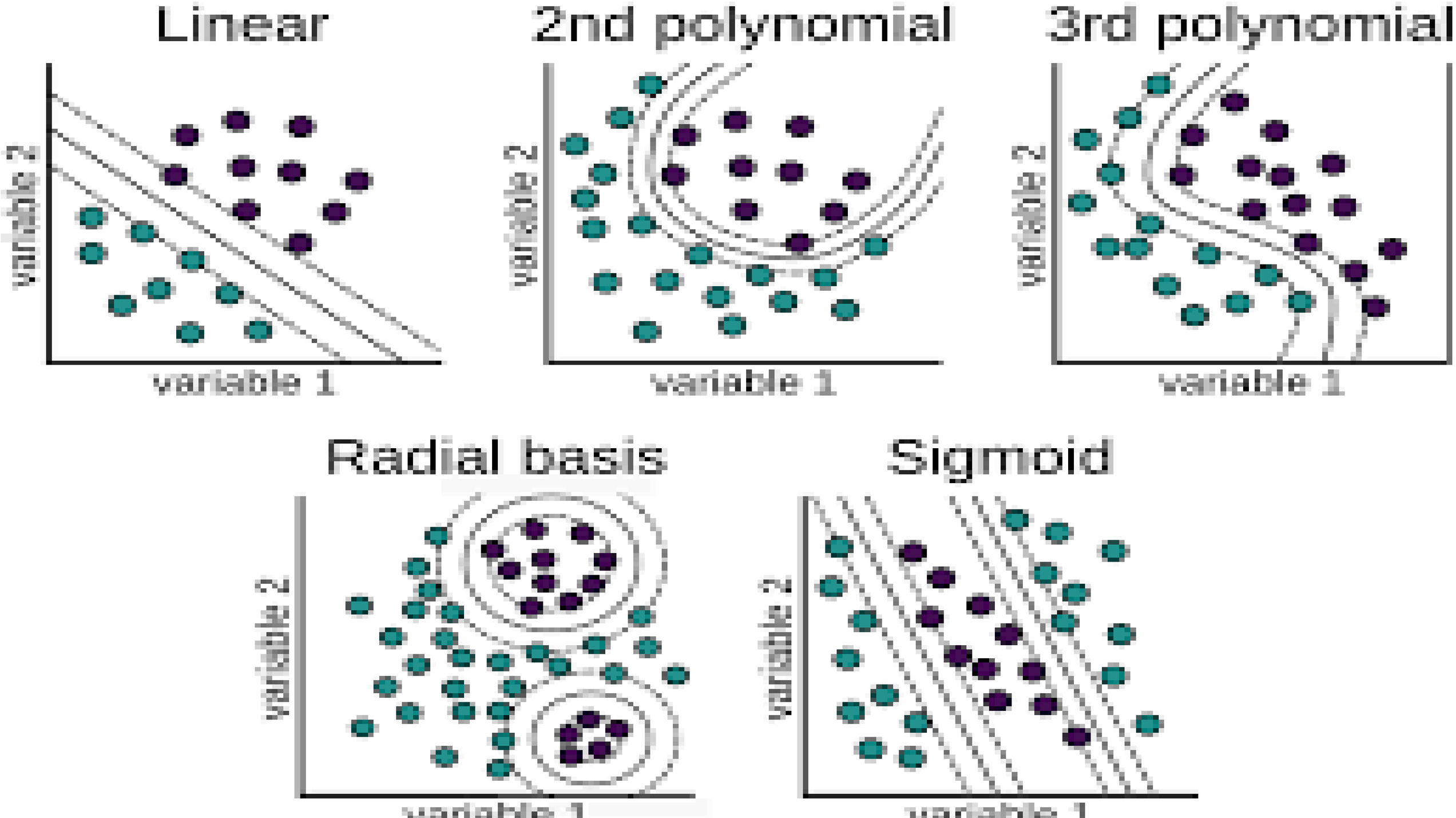


Input Space

(a)



(b)


Linear
variable 2
variable 1
2nd polynomial
variable 2
variable 1
3rd polynomial
variable 2
variable 1
Radial basis
variable 2
variable 1
Sigmoid
variable 2
variable 1

1) Strong Generalization Ability

SVM is designed to **maximize the margin**, which directly improves its ability to perform well on unseen data.

- Larger margin → lower overfitting risk
- Based on principles from Statistical Learning Theory

2) Works Well in High Dimensions

SVM performs effectively when:

- Number of features is very large
- Data is sparse (e.g., text classification)

This makes it useful in domains like:

- NLP
- Bioinformatics

3) Effective with Small to Medium Datasets

Unlike deep learning models, SVM:

- Does not require massive data
- Can achieve high accuracy with limited samples

4) Handles Non-Linear Data (Kernel Trick)

SVM can model complex patterns using kernels.

- Maps data into higher-dimensional space
- Avoids explicit computation (efficient)

Common kernels:

- Linear
- Polynomial
- RBF

This flexibility is a major reason for its importance.

5) Robust to Overfitting

SVM focuses only on **support vectors**, ignoring less critical points.

- Reduces sensitivity to noise (to some extent)
- Controlled using parameter **C**

THANK YOU